

Buster vs Claude's C Compiler

Measured native compilation, generated code, semantics, architecture, and engineering priorities.

29 September 2026 | Pinned-source comparison | Exploratory hosted measurements

Buster wins compilation throughput on the larger tested inputs. CCC wins scalar code size and the observed scalar runtime medians. Buster wins native SIMD code generation in the vector probe, debug output, and included validation infrastructure. There is no universal winner.

Decision	Evidence
Fast builds of substantial C units	Buster FAST: 2.30x faster for cJSON; 3.71x for macro functions; 6.32x for 4,096 functions.
Tiny compilation units	CCC: 2.84 ms vs Buster 20.71 ms for four functions. The Buster range is noisy: 13.97-26.98 ms.
Scalar generated programs	CCC: 1.09-3.65x faster observed medians across nine cells; cJSON 1.18x. Small differences need exclusive-host confirmation.
Scalar .text size	CCC: 27.8-81.6% smaller across all eight compile workloads.
Native 512-bit vector add	Buster 36 B vs CCC 610 B: 94.1% less .text, one native vpaddq. No vector runtime timing.
Peak compiler RSS	CCC wins seven cells; Buster uses 41.3% less RSS on 4,096 functions.
Reliability conclusion	All chosen runtime checks pass. This does not establish general C conformance or a compiler-wide correctness percentage.

Buster revision: e642c1e32e40568c168d36b0f281c76e4c939139

CCC revision: 6f1b99acb2f4ec2414592136c2009fe7713deec3

The Buster comparison uses its current C-only native compiler, not the removed custom-language frontend and not an LLVM optimizing backend. Its LLVM bitcode output is a distinct path. CCC means Anthropic's anthropics/claudes-c-compiler, not Clang or Claude Code itself.

Host limit: EPYC 9V74 shared virtual machine, x86-64 Linux, with AVX-512 exposed. This is not Ryzen 9700X / Zen 5 acceptance evidence. Results apply to the recorded binaries, inputs, flags, and host.

Compilation time and scaling

Median wall time in milliseconds. Seven rotated-order observations after warmup, process creation included; no compiler build runs inside the observations. Winner column compares Buster FAST with CCC. QUALITY and Clang are separate context.

Input	Buster FAST	QUALITY	CCC	Clang 20	Winner
4 functions	20.71	17.04	2.84	34.12	CCC 7.29x
32 functions	15.18	15.22	6.22	52.11	CCC 2.44x
512 functions	27.22	30.52	69.92	369.13	Buster 2.57x
4,096 functions	109.04	114.79	689.32	2,637.29	Buster 6.32x
1,024 macro functions	69.30	71.08	257.34	731.58	Buster 3.71x
cJSON 1.7.19	37.62	36.81	86.53	350.35	Buster 2.30x
cJSON driver	15.92	15.43	4.76	40.44	CCC 3.34x
8-kernel runtime TU	15.17	15.33	9.29	72.23	CCC 1.63x

What the scaling reveals

Buster's fixed process cost dominates very small units. At 512 and 4,096 functions, its lower marginal work cost reverses the result. Across 32 to 4,096 functions, the observed wall increment is 93.86 ms for Buster versus 683.09 ms for CCC. This is a measured difference for this generator, not a general asymptotic-complexity proof.

CCC performs richer SSA transforms, emits text assembly, applies text peepholes, and reparses assembly into ELF. Buster emits native machine representations directly and applies a bounded canonical FAST pipeline. These structural differences explain plausible cost sources, but the observed ratios are end-to-object results; no fraction of the difference is attributed to a phase without a matched diagnostic.

Action when Buster loses small-TU compilation

Profile an immutable binary on empty, four-function, and 32-function units. Separate loader/process time, eager table initialization, predefined symbols, arena setup, source discovery, and backend preparation. Initialize only data demanded by the selected operation. Reuse a persistent compiler or batch native translation units when the surrounding build can do so. No specific startup culprit was proved here.

Target the observed floor before further lexer SIMD work: four-function Buster time needs an 86.3% reduction to match the current CCC median, while 32 functions need 59.0%. Those percentages are competitive gaps, not predicted gains from a proposed patch.

Peak memory and scalar code size

RSS values are median MiB from three separate C fork/exec/wait4 captures. The original Python wrapper had an inherited pre-exec memory floor; original samples are retained, and accepted wall timings are unchanged. Code bytes are the sum of emitted .text sections, excluding debug data.

Input	Buster RSS	CCC RSS	Buster .text B	CCC .text B	CCC smaller
4 functions	22.82	8.25	694	295	57.5%
32 functions	22.92	9.30	5,622	2,309	58.9%
512 functions	30.26	25.99	90,102	37,623	58.2%
4,096 functions	89.34	152.22	720,886	300,933	58.3%
1,024 macro functions	64.66	62.45	507,889	93,696	81.6%
cJSON 1.7.19	36.16	23.38	54,777	39,571	27.8%
cJSON driver	23.10	9.11	1,167	605	48.2%
8-kernel runtime TU	23.12	9.91	6,892	4,479	35.0%

Memory winner depends on input size

CCC's four-function peak is 8.25 MiB against Buster 22.82 MiB. On cJSON it uses 23.38 MiB against 36.16 MiB. On 4,096 functions the order reverses: Buster 89.34 MiB against CCC 152.22 MiB, a 41.3% reduction. The macro cell's 3.4% RSS difference is small and should not be overgeneralized.

Code size is the larger Buster weakness

CCC reduces cJSON .text by 27.8% and macro-function .text by 81.6%. Even the simple exported scalar functions are about 58% smaller. A four-function case emits 694 B in Buster, 295 B in CCC, and 142 B in Clang. This is code-section size, not executable disk size or instruction-cache miss measurement.

Disassembly of f0: Buster 166 B versus CCC 73 B. Buster retains stack homes and generic shifts/multiplication where CCC uses smaller instruction forms. Prioritize immediate shifts, legal LEA patterns, memory operands, copy elimination, and unnecessary leaf-frame removal before adding an expensive global optimizer.

For RSS, collect allocation-by-phase and per-site counters separately from timing. Investigate retained source/token/IR pools and startup allocations before changing arena policy. Reuse bounded scratch; release per-TU state at safe ownership boundaries. A smaller reservation is not necessarily smaller RSS.

Generated-program runtime

Milliseconds, five rotated observations after warmup. Each row uses identical source, input seed, and fixed row-specific repetition count across compilers. Arrays are small; the indexed-memory cell is not a DRAM/cache-miss benchmark. All timed results match the Clang oracle.

Kernel	FAST	QUALITY	CCC	Clang 20	CCC faster
Integer reduction	1,510.59	1,498.43	602.25	56.60	2.51x
Integer division	104.48	102.14	68.53	53.61	1.52x
Loop invariants	361.51	359.65	145.59	43.24	2.48x
Repeated expressions	458.84	461.94	205.70	64.48	2.23x
Small-helper calls	455.86	451.74	239.09	34.35	1.91x
Dependent branch	124.33	108.67	34.06	54.43	3.65x
Indexed memory	223.68	221.05	125.37	51.95	1.78x
Floating arithmetic	190.20	180.48	174.70	53.12	1.09x
cJSON parse/print	119.76	124.36	101.10	57.06	1.18x

CCC has the lower observed FAST-comparison median in every row. The small floating-point gap needs stronger host control; all results are exploratory, without simultaneous confidence bounds or an admitted global performance verdict. CCC even beats Clang in the particular dependent-branch cell; that is not a general claim of superior optimization.

QUALITY is not the remedy demonstrated here

FAST and QUALITY produce whole-file identical objects for every synthetic TU, including the eight-kernel runtime TU. Their linked runtime .text is byte-identical. The timing differences therefore do not demonstrate allocator-generated code improvements. For cJSON, QUALITY saves 745 code bytes (1.36%) but has no demonstrated runtime benefit in this capture.

Current native path, not fallback artifacts

Separate untimed Buster profiles report zero fallback functions and zero runtime fallback locals for the tested native path. The runtime gaps belong to actual FAST code generation. Buster's canonical fallback remains implemented in the repository; this report does not claim native retirement is complete.

Eight synthetic kernels were checked over three seeds and three repetition counts before timing. cJSON has separate parse/print oracle checks. These successful observations are a bounded correctness result, not GCC torture, PostgreSQL regression, kernel boot, or whole-compiler conformance evidence.

Where CCC makes better scalar code

Loss / measured CCC advantage	Observed implementation difference	Buster remedy
Helpers: 1.91x faster	Buster retains two helper calls per inner iteration; CCC inlines them.	Add explicitly selected, budgeted tiny-function inlining; preserve argument evaluation, ABI, recursion/SCC limits, debug provenance.
Loop invariants: 2.48x faster	Buster repeats $x*y$, $x+y$, and the seed mask in the inner loop; CCC hoists them.	Build reusable loop/dominance facts. Hoist proven pure operations; do not hoist potentially trapping or memory/atomic/FP-observable operations by default.
Division: 1.52x faster	Buster executes both divisions; CCC replaces unsigned /37 with magic multiply and shift.	Implement target-lowered constant unsigned division with independent wide-integer equivalence tests and cost checks.
Branch cell: 3.65x faster	CCC uses cmovne for the tested diamond; Buster retains conditional branches.	Select branchless operations for short, side-effect-free diamonds using target cost and register-pressure limits; do not convert every branch.
Repeated expressions: 2.23x faster	CCC has GVN/copy propagation; Buster's four-pass canonical FAST pipeline has no general value numbering.	Begin with bounded block-local value numbering, then sparse propagation/constant-branch cleanup. Treat memory and FP identities conservatively.
Reduction: 2.51x faster; memory: 1.78x	Observed overall code-quality gaps; no isolated single-cause experiment. Clang remains much faster in the reduction cell.	Inspect loop control, scalar spills and missed simplification. After scalar cleanup, evaluate a narrow opt-in reduction vectorizer with tail, overflow and alias tests.

Preserve Buster's compile-throughput advantage

Keep FAST as a deliberate fast tier. Add a distinct optimizing tier rather than making every compile run a CCC-sized pipeline. Attach work/retained-memory limits, collect pass changes and costs outside timed runs, and skip clean functions. Reuse CFG/liveness facts with explicit invalidation. CCC's dirty-function and pass-dependency scheduling is a useful architectural example.

The measured 1.5-3.7x runtime gaps are reference targets for this corpus. They are not predictions that any one pass will deliver that gain. Require unchanged semantic checks plus matched compilation-time, RSS, code-size and runtime observations for each candidate.

SIMD and Zen 5

Dimension	Buster	CCC	Verdict
Compiler implementation	Compact AVX-512 lexer/translation kernels, packed arena/index-based representations.	String-based preprocessing, token ownership, SSA vectors/maps, assembly text roundtrips.	Buster has stronger SIMD-oriented design; measured large-input throughput favors Buster on this host.
Header-free 64-byte vector add	One native ZMM vpaddq; .text 36 B.	Eight scalar lane additions; .text 610 B.	Buster: 94.1% less code (16.94x size ratio), no runtime ratio measured.
Stock immintrin.h test	Clang 20 header inclusion fails on unrelated __builtin_ia32_clui.	Bundled headers compile but the requested AVX-512 operation scalarizes.	CCC wins this inclusion test; Buster wins native code generation once reachable.
CPU feature selection	Recognizes znver5 target model; true mask bank k1-k7.	No x86 tuning model found; x86 -march lacks tuning; CPU-support builtin always false.	Buster capability advantage. Actual 9700X performance remains unmeasured.
Loop autovectorization	No native general loop vectorizer found.	No native general loop vectorizer found.	Neither wins a demonstrated auto-vectorization capability.
Self-built compiler SIMD	Compact lexer guard excludes __BUSTER__; trusted Clang build uses the SIMD path.	C compiler written in Rust; C self-compilation is not applicable.	Buster self-built stages need separate profiling; trusted-host result cannot be transferred.

High-priority Buster improvements

Repair the stock intrinsic-header barrier tracked in [#1419](#). Register precise builtin signatures so unused supported header wrappers can be analyzed; give structured errors when an unsupported operation is actually required. Freeze and validate a resource-header version, or distribute a complete supported header set. Avoid scalar stubs with wrong intrinsic semantics.

For the vector probe, Clang emits 22 B versus Buster 36 B by using a memory operand and omitting the frame. Those are concrete selector/prologue opportunities: a 38.9% code-size gap to Clang in one function. Add real runtime vector checks before claiming corresponding throughput gains.

Schedule an exclusive 9700X comparison with native-CPU host builds, scalar and wide-vector workloads, multi-TU scaling, cache-capacity crossings, and precise feature/OS state. Four wide execution lanes do not make parsing, linking, irregular control flow, or cache misses disappear.

C semantics and frontend compatibility

Aspect	Finding	Winner / limitation
C/C++ language scope	Both current projects are C compilers. Buster's custom frontend is dormant. Neither is a C++ compiler.	Tie in active language scope.
ISO conformance / GNU coverage	Both have substantial C/GNU implementations and limitations; Buster has C23 subset support, CCC shallow -std handling. No common exhaustive conformance suite was run.	No global correctness or standards winner.
Implicit C11 atomics	Buster emits lock cmpxchg for _Atomic counter++; CCC plain load/add/store, matching qualifier erasure in source. Explicit CCC atomic builtins are a separate implemented path.	Buster wins this semantics probe. No performance comparison against a non-atomic implementation.
GNU error attribute	Clang diagnoses. Buster accepts source and preserves external call; CCC accepts and removes call.	Neither enforces the required diagnostic in the probe. Buster preserves the reference.
Fortify request	Buster preserves -D_FORTIFY_SOURCE=2 macro; CCC explicitly undefines it.	Buster wins request preservation; full fortify-wrapper support not established.
CPU dispatch builtin	Buster rejects __builtin_cpu_supports; CCC accepts but falsely reports AVX512F unavailable; oracle reports available.	Buster needs implementation; CCC's apparent acceptance is misleading.
Wide floating types	Buster binary16 runtime exists; bfloat16 runtime refused; AArch64 binary128 transport/widening exists but arithmetic/comparison/narrowing remain unsupported. CCC has ARM/RV binary128 libcalls, while x86 _Float128 aliases long double.	Coverage is split. CCC has source-implemented AArch64/RV wide arithmetic breadth, not a verified execution win here.
Intrinsic headers / host resources	Buster consumes an embedded host resource-header path. CCC ships 17 intrinsic header files, often scalar-emulated.	CCC wins self-contained header packaging; operation semantics must be checked.

Buster remedies: implement CPU feature queries with compiler/target/OS-state semantics; diagnose GNU error attributes at referenced calls; close the validated header surface; add binary128 runtime helper lowering with the exact target ABI and fenv/NaN tests. Do not discard semantic checks or advertise unsupported features merely to make configure scripts pass.

Targets, artifacts, debugging, and tooling

Aspect	Buster	CCC	Verdict
Native architecture targets	x86-64 and AArch64.	x86-64, i686, AArch64, RV64.	CCC 4 vs 2 native architecture targets; +i686 and RV64.
Host / object platform coverage	Linux/macOS/Windows and additional mobile/UEFI targets; ELF64/COFF/Mach-O.	Linux-developed ELF compiler; Windows/macOS not validated in supplied project.	Buster: 3 object families vs 1, broader OS/ABI infrastructure.
Native toolchain independence	Own native codegen, encoders, object/link/JIT modules. No LLVM dependency for timed native path.	Own native codegen, assembler/linker; Cargo manifest zero dependencies.	Both native paths are custom; CCC does not win by a presumed Buster LLVM dependency.
Dynamic linking breadth	Own -shared/-pie image path limited to Linux x86-64 in driver.	ELF shared/link implementations for four native backends.	CCC broader source-implemented ELF CPU coverage; no four-target execution sweep here.
Non-native output	Direct Wasm64 and LLVM bitcode, plus specialized paths with explicit limits; GPU orchestration distinct.	No matching backend output surface found.	Buster breadth advantage; output existence is not target-runtime proof.
Debugging / unwind	Own DWARF4, CodeView/PDB and unwind machinery; probe .debug_info/.debug_line/.eh_frame present.	Default standalone assembler discards debug/CFI directives; probe has no corresponding sections.	Buster wins emitted debug/unwind evidence; section presence is not a full debugger test.
CLI integration	ide cc subcommand; tested cc --version and --help rejected. Unsupported options generally diagnosed.	Standalone ccc / per-arch binaries; GCC version/triple interface. Unknown -f/-m flags silently ignored.	CCC wins packaging/interface; Buster wins explicit unsupported-option feedback.
Parallel compile scheduling	Opt-in -fcompile-jobs=N persistent native multi-input link lane gang; default 1.	Input sources compiled sequentially within invocation. Build system may parallelize processes.	Buster has internal scheduling capability; no scaling speedup measured.
Profiling interfaces	Source work census, per-pass timing, allocation/PMU/throughput tools and provenance.	CCC_TIME_PHASES/TIME_PASSES, change counts, pass disabling.	Both useful. Buster stronger performance-admission infrastructure.

Add a small buster-cc entry point, real version/triple query responses, relocatable builtin resources, and representative configure/CMake smoke tests. These address the measured interface gap without changing the optimizing pipeline. New i686/RV64 backends are larger investments; for your Zen 5 priority, current runtime/startup/header gaps have a more direct payoff.

Validation, maturity, and maintainability

Aspect	Evidence	Verdict
Included validation infrastructure	Buster includes desktop OS/architecture CI, sanitizer/fuzz, differential, ABI, metamorphic, self-host, strict fallback and pinned compatibility harnesses. CCC has 499 in-source test declarations; documented tests/ and .github/ trees absent at pinned checkout.	Buster infrastructure/reproducibility winner; not proof of greater intrinsic correctness.
Reported ecosystem breadth	Anthropic historical report describes Linux 6.9, QEMU, FFmpeg, SQLite, PostgreSQL, Redis and GCC torture. Current CCC tracker has 285 rows with failures, incomplete targets and changed time budgets. Buster has 13 pinned compatibility harnesses identified in audit.	CCC wins reported external-project breadth. The reports were not independently reproduced in this comparison.
Self-host validation	Buster implements a full-unity two-generation byte-identical fixed-point gate, not rerun in this comparison. CCC is Rust-written, so a C compiler cannot compile its implementation language.	Buster has applicable C self-host gate; no language-independent correctness ratio follows.
Compiler source footprint	CCC Rust: 351 files, 186,696 physical lines, 8,154,079 B. Buster compiler C/H: 130 files, 383,469 lines, 30,723,563 B; excluding generated files, 111 files/283,281 lines/14,281,451 B. Shared Buster foundations excluded.	CCC smaller physical compiler-source footprint. Scope/language/generated data prevent a maintainability score.
Memory-safety design	CCC source is safe Rust by inspection (no executable unsafe blocks found); Buster uses C arenas, indices and validation contracts. Neither compiler implementation was audited for security here.	CCC has language-level memory-safety advantage; no measured defect/security rate.
Licensing and reuse clarity	CCC declares CC0-1.0. Buster README explicitly says third-party notices do not grant a license to first-party code.	CCC clearer external reuse terms; Buster needs an intentional first-party licensing decision if external adoption is desired.
Maintenance risk	CCC has a richer transform surface and publicly warned unvalidated AI-authored documentation; Buster has larger dense modules, generated metadata and many invariant/admission mechanisms.	No objective universal winner. Auditability depends on clear contracts and reproducible tests, not LOC or test-count ratios.

Do not divide Buster's millions of assertions by CCC's 499 test declarations, or compare either to a historical 99% torture result. Those denominators differ. The common-input checks in this run are the directly comparable correctness evidence.

Buster's validation expansion should target pristine pinned GCC torture, Linux kernel configurations with boot tests, PostgreSQL regressions and FFmpeg checkasm. Keep unsupported cases and failed checks visible, and retain independent execution evidence. That closes the most useful CCC ecosystem advantage.

Recommended order for Buster

Priority	Concrete work	Success evidence
1. Startup and RSS floor	Attribute fixed small-TU costs; make unrelated tables/resources lazy; bounded persistent/batched compilation where appropriate.	Four/32-function wall and peak RSS improve; cJSON/large-input throughput stays within predeclared budgets. No guessed initialization culprit.
2. Cheap native instruction selection	Immediate shifts, LEA/multiply idioms, legal memory operands, redundant-copy removal, leaf frames, lock xadd for simple equivalent fetch-add.	Exact disassembly changes plus semantic regressions, ABI/self-host validation, code-size and compile-time measures.
3. Reach existing SIMD through headers	Implement validated builtin signatures and referenced-operation diagnostics; close #1419; freeze the resource interface.	Stock immintrin test compiles; genuine wide instructions and runtime lane/mask/FMA semantics match an independent oracle.
4. Explicit optimizing tier	Bounded tiny inlining, local value numbering/SCCP/constant CFG cleanup, constant division, loop-invariant motion, careful if-conversion. Reuse analyses and dirty worklists.	Each candidate isolates its effect on relevant lost runtime cell and documents added compile/RSS cost. Keep FAST default.
5. Precise frontend contracts	Implement CPU-support query and GNU error attribute enforcement. Add binary128 runtime lowering when target demand justifies it.	The concrete probes stop rejecting supported operations or silently accepting unenforced semantics; target ABI/fenv validation for wide FP.
6. Standalone distribution	buster-cc facade, real version/triple queries, bundled/versioned resources, chosen first-party reuse terms.	Stock configure/CMake/build workflows consume an installed compiler without monorepo path assumptions.
7. Wider real-source validation	Pin GCC torture, Linux boot configurations, PostgreSQL and FFmpeg oracles.	Pristine builds and full target executions with exact revisions, retained logs, explicit failure/unsupported inventories.
8. Exclusive Zen 5 follow-up	Run frozen artifacts on 9700X; measure startup/large TUs, wide SIMD, multi-TU scaling, cache-capacity crossings and sustained mixed kernels.	Noise controls, A/A, exact binary/input/environment hashes and matched source/host-build controls. No VM result promoted to Zen 5 admission.

Decision for your current goals: Buster is already the stronger choice for fast substantial C compilation on the measured configuration. CCC supplies concrete optimization techniques and ecosystem targets to learn from. First remove fixed overhead and poor instruction forms; then offer runtime optimization as an explicit, budgeted choice. A broad optimizer by itself would not guarantee better overall time to an artifact.

Reproduction and interpretation limits

Builds and operation boundaries

Buster: Clang 20.1.2 Release, unity, tests off, host -O3/-march=native; hosted Clang build-driver exception, default native linker selection. CCC: Rust 1.90.0, stock cargo release --bin ccc --offline, no GCC assembler/linker features. These are different projects' release host builds, not identical compiler-host instruction targets.

Common timed C flags: -O2 -std=gnu17 -fno-pic. Buster adds -g0 because debug defaults on; QUALITY adds -fregister-allocator=quality. CCC omits all -g options because its parser interprets -g0 as enabling debug. Each compiler uses its own optimizer; -O2 does not imply identical effort or codegen features.

Artifact target defaults also differ: the Buster profile reports cpu=znver4 and vector_bits=512, while CCC and Clang use their default x86-64 configurations. This compares the recorded native defaults, not an identical CPU-feature/cost-model sweep. Buster profiles report no native/split SIMD operations for the scalar corpus; Clang uses SSE2 in its reduction. Host compiler ISA and emitted-program ISA are distinct.

Compilation observations end at objects. Runtime observations link those objects using the same Clang 20 -no-pie driver to control linker/runtime differences. Separate untimed probes confirm each compiler's own standalone linking on no-header and stdio programs. Large-project link speed, LTO, runtime loader cost and multi-target execution are not measured.

Inputs and sampling

Four deterministic exported scalar-function sizes, one 1,024-function macro generator, the eight-kernel TU, stock cJSON 1.7.19 and its small driver. cJSON pin c859b25da02955fef659d658b8f324b5cde87be3. The library source is unchanged and its MIT license is included. Unsigned arithmetic avoids signed-overflow UB; floating operations use exact binary fractions. Checksums are consumed and checked before and during timing.

Parent and child affinity CPU0; shared host activity remains possible. Seven rotating compile samples, five rotating runtime samples, no overlapping timed workers. Raw minima/maxima, compiler/input SHA-256, command argv, validation outputs and failures are retained. Three independent C wait4 RSS observations repair the Python pre-exec floor; the original data remain available.

What remains unranked

General C standard compliance; broad optimizer soundness; security defect rates; Zen 5 throughput; instruction-cache/DRAM behavior; compilation/function/TU concurrency scaling; full Linux/kernel/PostgreSQL/FFmpeg compatibility; wide-FP target execution; linker performance on real projects; debugging variables/stepping end-to-end. Source capability comparisons and historical reports are explicitly distinct from measured results.

No universal speedup is obtained by averaging unrelated dimensions. The optional equal-weight runtime geometric mean in summary.json is only a descriptive statistic for this selected corpus and is not used to declare a general winner.

Frozen raw_results.json SHA-256:

5ae9e6a51ebc228ca26ddc5973eef64e36eb1e2acc252affb4639bc105cc6f2c

Primary source map and evidence package

All source claims refer to the pinned commits above. Local analysis notes include precise symbol/line references; links below point to the inspected implementations. The February engineering article is historical primary reporting, not current measured proof.

1. [Buster active compiler catalogue](#)
2. [Buster canonical FAST pass contract](#)
3. [Buster FAST transforms](#)
4. [Buster driver / allocator / target behavior](#)
5. [Buster CLI and multi-TU scheduling](#)
6. [Buster atomic type and access contracts](#)
7. [Buster wide-float boundary](#)
8. [Buster compact lexer guards](#)
9. [Buster build and self-host contracts](#)
10. [Buster validation/CI](#)
11. [Buster reproducible performance harness](#)
12. [CCC optimizer orchestration and analysis reuse](#)
13. [CCC register allocation](#)
14. [CCC option handling](#)
15. [CCC source pipeline and fortify erasure](#)
16. [CCC atomic qualifier erasure](#)
17. [CCC scalar AVX-512 header](#)
18. [CCC debug directive dropping](#)
19. [CCC CPU support builtin](#)
20. [CCC GNU error call lowering](#)
21. [CCC current project claims and disclaimer](#)
22. [CCC project tracker \(mixed outcomes\)](#)
23. [Anthropic historical compiler engineering article](#)
24. [Buster intrinsic-header compatibility issue #1419](#)

Package contents

Buster_vs_Claude_C_Compiler_evidence.zip contains generated and stock workload sources, benchmark.py, C wait4 RSS observer, raw_results.json, summary.json, all feature probes and disassembly, untimed profiles, measurement notes, pinned-source audit notes, a build/reproduction README, and this report generator. Compiler binaries and repository clones are omitted; rebuild from the pinned repositories.